

API de Consulta de CPF Documenta o

API de Consulta de CPF

Visão Geral

API REST em Go para consulta de CPF com autenticação via Bearer Token. A API valida o CPF localmente (formato e dígitos verificadores) antes de consultar a API pública da Receita WS.

Pré-requisitos

- Go 1.21+
- Docker e Docker Compose (para execução containerizada)
- Conta e token de acesso próprio (variável `API_TOKEN`)

Instalação

```
git clone <repositório>
cd RafMachine

# Copiar e configurar variáveis de ambiente
cp .env.example .env
# Edite .env e defina API_TOKEN com um valor seguro
```

Como Rodar Localmente

```
# Instalar dependências
go mod download

# Rodar o servidor
go run ./cmd/api
```

O servidor inicia na porta definida em `PORT` (padrão: 8080).

Como Rodar via Docker

```
# Build e start
docker compose up --build -d

# Verificar status
docker compose ps

# Ver logs
docker compose logs -f api

# Parar
docker compose down
```

Endpoint

GET /consulta/cpf/{cpf}

Consulta os dados de um CPF na Receita Federal.

Autenticação: Bearer Token obrigatório no header Authorization.

Parâmetros de Path

Parâmetro	Tipo	Descrição
cpf	string	CPF com 11 dígitos numéricos (aceita formato 000.000.000-00)

Exemplos de Requisição e Resposta

Sucesso — 200 OK

Request:

```
GET /consulta/cpf/52998224725 HTTP/1.1
Host: localhost:8080
Authorization: Bearer supersecret123
```

Response:

```
{
  "cpf": "529.982.247-25",
  "nome": "JOAO DA SILVA",
  "situacao": "REGULAR",
  "data_nascimento": "01/01/1990",
```

```
    "digito_verificador": "25"
  }
```

Também aceita CPF formatado na URL:

```
curl -H "Authorization: Bearer supersecret123" \
http://localhost:8080/consulta/cpf/529.982.247-25
```

CPF Inválido — 400 Bad Request

Request:

```
GET /consulta/cpf/12345 HTTP/1.1
Authorization: Bearer supersecret123
```

Response:

```
{
  "error": "invalid CPF: must be 11 digits with valid check
digits"
}
```

Casos que geram 400: - Menos ou mais de 11 dígitos - Todos os dígitos iguais (ex: 11111111111) - Dígitos verificadores inválidos - Caracteres não numéricos (após remoção de . e -)

Não Autorizado — 401 Unauthorized

Request sem token:

```
GET /consulta/cpf/52998224725 HTTP/1.1
```

Response:

```
{
  "error": "authorization token required"
}
```

Request com token inválido:

```
GET /consulta/cpf/52998224725 HTTP/1.1
Authorization: Bearer token-errado
```

Response:

```
{
  "error": "invalid authorization token"
}
```

CPF Não Encontrado — 404 Not Found

Response:

```
{
  "error": "CPF not found"
}
```

Serviço Externo Indisponível — 503 Service Unavailable

Response:

```
{
  "error": "external CPF service is currently unavailable"
}
```

Retornado quando a API externa está fora do ar ou o timeout de 10s é atingido.

Erro Interno — 500 Internal Server Error

Response:

```
{
  "error": "internal server error"
}
```

Health Check — 200 OK (sem autenticação)

GET /health HTTP/1.1

Response:

```
{"status": "ok"}
```

Tabela de Códigos de Erro

HTTP Status	Código	Situação
200	OK	Consulta realizada com sucesso
400	Bad Request	CPF com formato inválido ou dígitos incorretos
401	Unauthorized	Token ausente ou inválido

HTTP Status	Código	Situação
404	Not Found	CPF não encontrado na base da Receita
503	Unavailable	API externa indisponível ou timeout
500	Server Error	Erro interno inesperado

Exemplos com curl

```
# Consulta com CPF em dígitos puros
curl -s \
  -H "Authorization: Bearer supersecret123" \
  http://localhost:8080/consulta/cpf/52998224725 | jq .

# Consulta com CPF formatado
curl -s \
  -H "Authorization: Bearer supersecret123" \
  "http://localhost:8080/consulta/cpf/529.982.247-25" | jq .

# Sem token (deve retornar 401)
curl -s http://localhost:8080/consulta/cpf/52998224725 | jq .

# CPF inválido (deve retornar 400)
curl -s \
  -H "Authorization: Bearer supersecret123" \
  http://localhost:8080/consulta/cpf/11111111111 | jq .

# Health check
curl -s http://localhost:8080/health
```

Exemplos com Postman

Configurar Collection

1. Crie uma nova Collection chamada **CPF API**.
2. Adicione uma variável de collection: `base_url` = `http://localhost:8080` e `token` = `supersecret123`.

Consultar CPF (sucesso)

- **Method:** GET
- **URL:** `{{base_url}}/consulta/cpf/52998224725`
- **Headers:**
 - Authorization: Bearer `{{token}}`

Testar 401

- **Method:** GET
- **URL:** {{base_url}}/consulta/cpf/52998224725
- (sem header Authorization)

Testar 400

- **Method:** GET
- **URL:** {{base_url}}/consulta/cpf/11111111111
- **Headers:**
 - Authorization: Bearer {{token}}

Variáveis de Ambiente

Variável	Obrigatória	Padrão	Descrição
API_TOKEN	Sim	—	Token Bearer para autenticação
PORT	Não	8080	Porta do servidor HTTP
RECEITA_WS_URL	Não	https:// www.receitaws.com.br/ v1/cpf	URL base da API externa de CPF

Documentação de Testes

Objetivo

Esta documentação descreve a estratégia de testes da API de Consulta de CPF, cobrindo testes unitários e de integração, como executá-los e como visualizar o relatório de cobertura.

Grupos de Testes

1. internal/validator — Testes Unitários do Validador de CPF

Arquivo: internal/validator/cpf_validator_test.go

Objetivo: Garantir que a função Validate aceita CPFs válidos e rejeita CPFs inválidos em todos os cenários possíveis, cobrindo as regras da Receita Federal.

Funções testadas: - Validate(cpf string) bool - FormatCPF(cpf string) string - StripCPF(cpf string) string

2. internal/middleware — Testes Unitários do Middleware de Autenticação

Arquivo: internal/middleware/auth_test.go

Objetivo: Verificar que o middleware BearerAuth permite apenas requisições com token válido e bloqueia todas as demais com 401 Unauthorized.

3. internal/handler — Testes de Integração do Handler

Arquivo: internal/handler/cpf_handler_test.go

Objetivo: Testar o handler Consultar com um mock da API externa, cobrindo todos os cenários de resposta (sucesso, não encontrado, API indisponível, erros internos).

A API externa é substituída por um mockCPFService que implementa a interface service.CPFService, isolando o handler de chamadas reais à Receita WS.

Como Executar os Testes

Localmente

```
# Rodar todos os testes
go test ./...
```

```
# Com output detalhado
go test ./... -v
```

```
# Com percentual de cobertura por pacote
go test ./... -cover

# Gerar arquivo de cobertura HTML
go test ./... -coverprofile=coverage.out
go tool cover -html=coverage.out -o coverage.html
# Abrir coverage.html no navegador
```

Rodar somente um pacote

```
go test ./internal/validator/... -v
go test ./internal/middleware/... -v
go test ./internal/handler/... -v
```

Rodar um teste específico

```
go test ./internal/validator/... -run TestValidate/all_ones
go test ./internal/middleware/... -run TestBearerAuth_MissingToken
```

Como Executar os Testes Dentro do Container Docker

```
# Build do container de teste
docker build --target builder -t cpf-api-test .

# Rodar os testes dentro do container
docker run --rm cpf-api-test go test ./... -cover

# Com output detalhado
docker run --rm cpf-api-test go test ./... -v -cover
```

Ou usando um Dockerfile de teste temporário:

```
docker run --rm \
-v $(pwd):/app \
-w /app \
golang:1.21-alpine \
go test ./... -cover
```

Visualizar Relatório de Cobertura HTML

```
# 1. Gerar arquivo de cobertura
go test ./... -coverprofile=coverage.out
```



```
# 2. Abrir relatório interativo no navegador (Linux)
go tool cover -html=coverage.out

# 3. Ou gerar arquivo HTML estático
go tool cover -html=coverage.out -o coverage.html
xdg-open coverage.html
```

Tabela de Casos de Teste

cpf_validator_test.go

#	Grupo	Caso de Teste	Entrada	Saída Esperada	Status
1	Válido	CPF 529.982.247-25	52998224725	true	PASS
2	Válido	CPF 111.444.777-35	11144477735	true	PASS
3	Válido	CPF 000.000.001-91	00000000191	true	PASS
4	Dígitos iguais	Todos zeros 000.000.000-00	000000000000	false	PASS
5	Dígitos iguais	Todos uns 111.111.111-11	111111111111	false	PASS
6	Dígitos iguais	Todos nove 999.999.999-99	999999999999	false	PASS
7	Tamanho errado	Muito curto (10 dígitos)	1234567890	false	PASS
8	Tamanho errado	Muito longo (12 dígitos)	123456789012	false	PASS
9	Tamanho errado	String vazia	""	false	PASS
10	Não numérico	Contém letras	1234567890A	false	PASS
11	Não numérico	CPF formatado (pontos e traço)	529.982.247-25	false	PASS
12	Não numérico	Contém espaços	529 982 247 25	false	PASS
13	Não numérico	Contém caractere especial @	52998224@25	false	PASS
14	DV inválido	Primeiro dígito verificador errado	52998224715	false	PASS

#	Grupo	Caso de Teste	Entrada	Saída Esperada	Status
15	DV inválido	Segundo dígito verificador errado	52998224726	false	PASS
16	DV inválido	Ambos dígitos errados	52998224700	false	PASS
17	Válido	Sequencial matematicamente válido	12345678909	true	PASS
18	DV inválido	Sequencial com DV errado	12345678901	false	PASS

auth_test.go

#	Caso de Teste	Header Authorization	API_TOKEN env	Saída Esperada	Status
1	Token válido	Bearer test-secret-token	test-secret-token	200	PASS
2	Token ausente	(sem header)	test-secret-token	401	PASS
3	Token inválido	Bearer wrong-token	test-secret-token	401	PASS
4	Bearer com valor vazio	Bearer	test-secret-token	401	PASS
5	Esquema errado (Basic)	Basic test-secret-token	test-secret-token	401	PASS
6	API_TOKEN não configurado	Bearer test-secret-token	""	401	PASS

cpf_handler_test.go

#	Caso de Teste	CPF na URL	Mock retorna	HTTP Status Esperado	Status
1	Consulta com sucesso	52998224725	*CPFData válido	200	PASS
2	CPF com formato inválido (curto)	1234	(não chamado)	400	PASS
3		11111111111	(não chamado)	400	PASS

#	Caso de Teste	CPF na URL	Mock retorna	HTTP Status Esperado	Status
	CPF com todos dígitos iguais				
4	CPF com DV errado	52998224700	(não chamado)	400	PASS
5	CPF não encontrado	52998224725	ErrNotFound	404	PASS
6	API externa indisponível	52998224725	ErrUnavailable	503	PASS
7	Erro interno inesperado	52998224725	errors.New(...)	500	PASS
8	Content-Type da resposta	52998224725	*CPFData válido	application/json	PASS
9	CPF formatado na URL	529.982.247-25	*CPFData válido	200	PASS

Metas de Cobertura

Pacote	Cobertura Atual	Meta Mínima
internal/handler	100%	80%
internal/middleware	100%	80%
internal/validator	94.6%	80%
internal/service	(integração)	—

Nota: O pacote internal/service não possui testes unitários autônomos pois depende de uma API HTTP externa. O comportamento do serviço é coberto indiretamente pelos testes de integração do handler via mock.